

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Implementación de Robótica Inteligente	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez Cesar Torres Huitzil	
Módulo:	Actividad: Mini Reto Semana 6	
Fecha: 18 de mayo del 2026		

Resumen

El presente reporte documenta el desarrollo e implementación de un sistema autónomo de seguimiento de línea sobre la plataforma Puzzlebot, complementado con la detección de un semáforo de tres colores que actúa como elemento de decisión externo dentro del comportamiento del robot. La solución se construyó sobre una arquitectura distribuida en dos nodos comunicados mediante ROS 2: un nodo de visión ejecutado sobre la Jetson Nano del robot, encargado de la captura y procesamiento de imágenes provenientes de la cámara CSI, y un nodo controlador ejecutado en una computadora externa, responsable de generar los comandos de velocidad lineal y angular publicados al tópico `/cmd_vel`. El nodo de visión aplica técnicas de preprocesamiento de imagen mediante la librería OpenCV, incluyendo la selección de regiones de interés, conversión a escala de grises, filtrado Gaussiano, umbralización binaria inversa y operaciones morfológicas de apertura y cierre para la segmentación de la línea, así como segmentación por color en el espacio HSV con filtros geométricos de área y circularidad para la detección del semáforo. El controlador implementa un esquema de tipo Pure Pursuit que calcula el error angular entre el punto de seguimiento y el centro de la imagen, integrado con una máquina de estados (GO, SLOW, STOP, NONE) que modula la velocidad del robot conforme al color detectado. Los resultados experimentales confirmaron que el Puzzlebot logra seguir de manera estable la línea central

de la pista y modificar correctamente su comportamiento en respuesta a las señales del semáforo, cumpliendo así con los objetivos planteados para el reto.

Objetivos

El objetivo principal de este reto es desarrollar e implementar un sistema autónomo de seguimiento de línea para un robot tipo Puzzlebot mediante el uso de visión artificial y ROS 2. Con este sistema se busca que el robot sea capaz de detectar una línea en la pista utilizando la cámara del Puzzlebot, procesar la información visual en tiempo real y generar los comandos de movimiento necesarios para mantenerse en el camino o trayectoria deseada de la pista.

Tomando en cuenta lo anterior, como parte de este reto se plantea implementar un nodo de visión en la Jetson Nano que permite procesar imágenes por la cámara del Puzzlebot. Para ello, el objetivo del reto es aplicar técnicas de preprocesamiento de imágenes como la selección de regiones de interés, filtrado, segmentación, operadores morfológicos, cálculo de centroides, entre otros. Lo anterior con el fin de obtener una referencia confiable de la línea dentro del entorno de navegación. Por consiguiente, otro de los objetivos del reto semanal es diseñar e implementar un controlador de seguimiento de línea que permita recibir información generada por el nodo de visión y calcular los comandos de velocidad lineal y angular que serán enviados al robot mediante el tópico `/cmd_vel`.

Finalmente, esta actividad semanal también tiene como objetivo integrar la detección de un semáforo dentro de la capa de decisión del robot. De esta manera el Puzzlebot debe modificar su comportamiento dependiendo del color detectado por la cámara. Consecuentemente, en este mini reto se pretende implementar estrategias de control y recuperación ante la pérdida temporal de una línea con el fin de que se mejore la robustez del sistema ante condiciones reales como el ruido de la imagen, cambios en la luz, curvas en pista o interrupciones.

Introducción

El presente reto plantea el desarrollo de un sistema autónomo de seguimiento de línea sobre la plataforma Puzzlebot, un robot diferencial equipado con una cámara CSI montada sobre una computadora embebida Jetson Nano. La pista de pruebas consiste en una superficie de MDF sobre la cual se encuentran tres líneas paralelas trazadas con cinta negra, donde se busca que el robot identifique la línea central y la siga de manera estable a lo largo de tramos rectos y curvos. A esta tarea base se incorpora un segundo objetivo que añade complejidad al

sistema: la detección de un semáforo de tres colores que actúa como elemento de decisión externo y modifica el comportamiento del robot conforme al estado del entorno, simulando así las condiciones que enfrentaría un vehículo en un escenario de tráfico controlado.

Para resolver este problema se desarrolló una arquitectura distribuida compuesta por dos nodos comunicados mediante el middleware ROS 2. El primero, ejecutado directamente sobre la Jetson Nano, concentra todo el procesamiento de visión: lectura de la cámara CSI, segmentación de la línea, detección del semáforo y publicación de las señales relevantes como tópicos. El segundo nodo, ejecutado sobre una computadora externa, consume dichas señales y aplica un controlador de tipo Pure Pursuit en conjunto con una máquina de estados asociada al color detectado del semáforo, generando los comandos de velocidad lineal y angular que se envían al robot mediante el tópico `/cmd_vel`. Esta separación física entre percepción y control responde tanto a consideraciones prácticas de carga computacional sobre la Jetson como a un principio de diseño modular que facilita la depuración y el ajuste independiente de cada componente del sistema.

La utilización de visión por computadora permite que el robot obtenga e interprete información de su entorno en tiempo real y a partir de ello se tomen decisiones, como en este caso a partir de lo observado del semáforo, y de la imagen pre procesada tanto para el semáforo como para las líneas de la pista. Este procesamiento de imagen se hizo con la librería OpenCV, que permite hacer operaciones de visión por computadora. Este procesamiento permite mantener detecciones más confiables e imágenes más limpias.

Solución del problema

La solución del reto se desarrolló mediante una arquitectura compuesta por un nodo de visión ejecutado en la Jetson del puzzlebot y un nodo controlador ejecutado en una de las computadoras de los miembros del equipo. El sistema integra detección de línea, detección de semáforo y control de movimiento para permitir que el robot siga una trayectoria de manera autónoma y responda a diferentes estados del entorno.

El sistema inicia con la obtención de la imagen captada por la cámara CSI con resolución de 1640×1232 píxeles. Posteriormente, el pipeline de visión procesa cada frame para extraer información relevante, publicando datos como el punto de seguimiento (*lookahead point*), visibilidad de línea y color detectado del semáforo. Finalmente, el nodo controlador utiliza esta información para calcular las velocidades lineales y angulares que se envían al robot mediante el tópico `/cmd_vel`.

Para la detección de líneas, primero se seleccionó una región de interés para reducir la carga computacional y concentrar el análisis únicamente en la zona donde se esperaba encontrar la línea. La imagen obtenida se convirtió de formato BGR a escala de grises con el objetivo de simplificar la información visual y trabajar únicamente con intensidad luminosa. Posteriormente se aplicó un filtro Gaussiano de tamaño 7×7 para reducir ruido de alta frecuencia y suavizar la imagen antes de la segmentación.

La segmentación de la línea se realizó mediante una umbralización binaria inversa. Este procedimiento permite distinguir la línea de color negro del fondo correspondiente a la pista de pruebas, generando una máscara binaria que se utilizará para el seguimiento. Después de la umbralización se aplicó una operación de apertura, definida como una erosión seguida de una dilatación. Esta operación eliminó pequeños puntos aislados y ruido residual presentes en la máscara binaria. Posteriormente se aplicó una operación de cierre, la cual permite cerrar huecos y discontinuidades en la línea segmentada. Ambas operaciones utilizaron un kernel morfológico de 3×3 , logrando una detección más estable durante el movimiento del robot.

Para la detección del semáforo se utilizó un método basado en segmentación por color en el espacio HSV. La conversión de BGR a HSV permite separar de mejor manera la información de tonalidad e iluminación, facilitando la identificación de colores bajo distintas condiciones de luz. Se generaron máscaras binarias independientes para los colores *verde*, *amarillo* y *rojo*. En el caso del rojo fue necesario utilizar dos rangos HSV debido a que el matiz de este color se encuentra en ambos extremos del espacio cromático, cerca de 0° y 180° .

En el caso del color amarillo, se implementó un filtro geométrico para reducir falsos positivos causados por detecciones de espacios con luz excesiva. Estos filtros se basaron en el área de píxeles que conforman este color, y en la circularidad de la misma. Esta última se calculó mediante la siguiente expresión: $C = \frac{4\pi A}{P^2}$, donde A es el área y P el perímetro. De esta forma, un objeto era considerado válido mientras superara un mínimo de área y circularidad. Este mismo filtro se utilizó para controlar la distancia a la que debía detectarse el semáforo, al establecer un mínimo de área detectable para modificar los estados de la máquina de estados presente en el controlador, misma que se explicará más adelante.

El control de trayectoria se resolvió mediante un controlador *Pure Pursuit*. Este método utiliza un punto llamado *lookahead point*, el cual sirve como referencia para orientar

el movimiento del robot. El controlador calcula el error angular tomando como referencia la diferencia entre la posición horizontal del punto detectado y el centro de la imagen:

$$dx = la_x - frame_width/2$$

Desplazamiento horizontal, donde la_x es la coordenada x del lookahead point, y $frame_width / 2$ representa el centro de la imagen y es la odometría geométrica del robot

$$dy = frame_height - la_y$$

Desplazamiento vertical, donde la_y es la coordenada y del lookahead point.

Utilizando estos dos valores anteriores, el ángulo de corrección para centrar el robot con respecto a la línea se calcula mediante la siguiente fórmula:

$$\alpha = \text{atan2}(dx, dy)$$

Cuando el valor de α aumenta, el robot incrementa su velocidad angular para corregir la desviación respecto a la línea.

El nodo controlador utiliza la información proveída por el nodo de la detección de semáforos y la corrección del ángulo de conducción para modificar el movimiento mismo del robot. para modificar la velocidad del robot en función del color de semáforos detectado, se utiliza la siguiente máquina de estados:

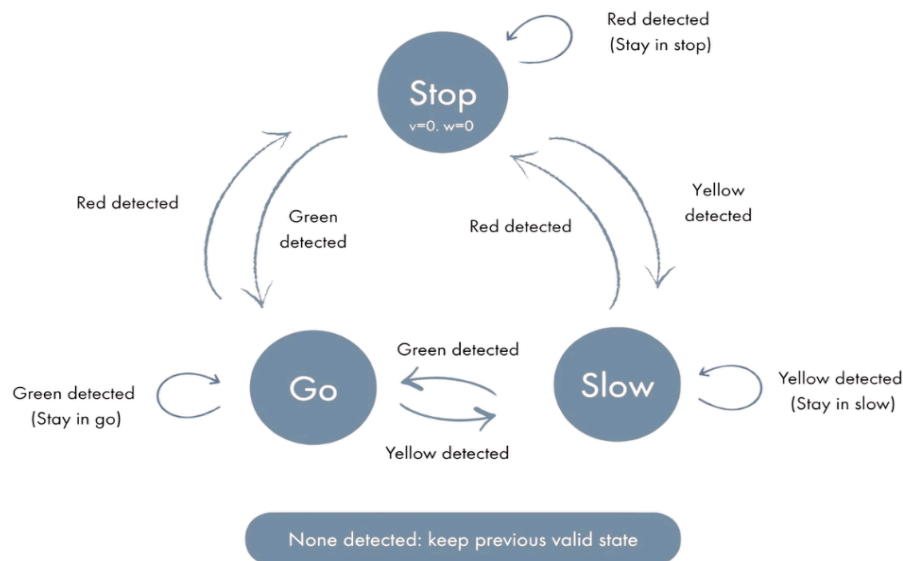


Imagen 1. Máquina de estados del nodo controlador.

A grandes rasgos, el nodo espera a que el robot detecte verde para comenzar con su funcionamiento, una vez detectado, el robot pasa al estado *GO* y se moverá a velocidad “normal” cuando detecte verde, y mientras siga detectado verde. Cuando el robot detecte amarillo, se pasa al estado *SLOW* y el nodo reducirá la velocidad del robot en preparación para cuando se detecte rojo. La velocidad permanecerá así mientras se siga detectando amarillo. Cuando se detecte rojo, se pasará al estado *STOP* y el robot se detendrá totalmente, permaneciendo así mientras siga detectando rojo. En el caso que no se detecte ninguno de estos 3 colores, la máquina de estados pasará a estado *NONE*, y permanecerá en el último estado marcado por el último color válido detectado.

Utilizando este estado, y el cálculo del valor de α , el nodo controlador publica distintos valores a la velocidad angular y lineal del robot al tópico de `/cmd_vel` del robot para modificar el movimiento del robot en función a la trayectoria que este debe tomar.

Resultados

Los resultados obtenidos fueron bastante satisfactorios y cumplieron con lo esperado. El Puzzlebot es capaz de seguir la trayectoria que observa desde la cámara que tiene integrada. Esto gracias al controlador geométrico que se ha implementado. La pista en la cual el puzzlebot realiza el camino consta de una base de MDF y tres líneas que forman el camino. Para este caso en específico, se utilizó únicamente la línea central, como se puede observar en la imagen 2.

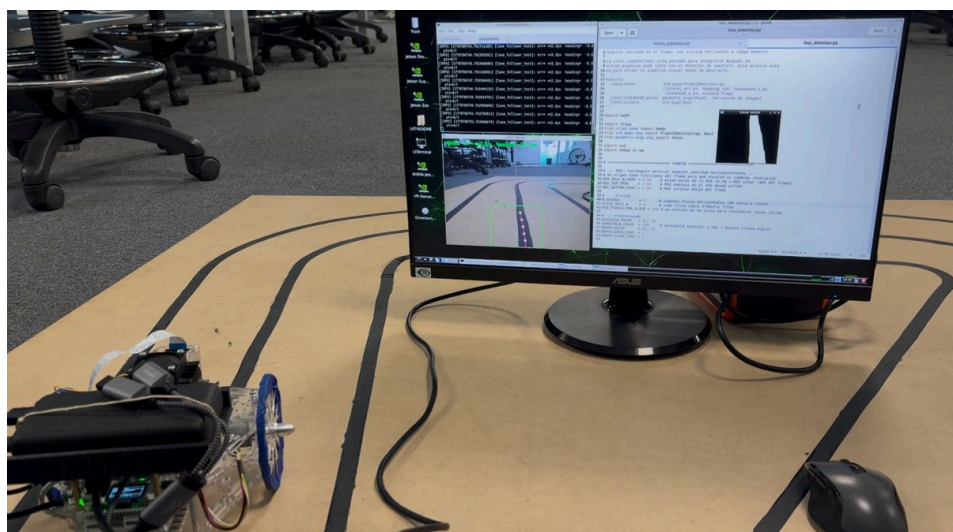


Imagen 2. Puzzlebot en pista y controlador pure pursuit corriendo.

Como se puede observar, el robot debe ser posicionado en la línea central, pues el controlador *pure pursuit* asume que ya está correctamente alineado a la línea a seguir y cuenta con una referencia en el robot para el punto *look ahead* del controlador geométrico y de esta manera corregir y seguir la línea de la manera esperada. Igualmente, en esta imagen se puede observar tanto lo que observa el robot como la imagen procesada de la ROI, es decir, la imagen que ha pasado por el cambio de BGR a escala de grises, el ruido Gaussiano, umbralización inversa y las operaciones de apertura y cierre.

Asimismo, se utilizó la información del tópico */detected_color* donde se recupera lo que observa con la cámara en la ROI designada para el semáforo, dependiendo del color, el comportamiento del robot cambia. Cuando el puzzlebot detecta verde el robot comienza a avanzar, o si ya lo hacía, sigue con ese movimiento. Es importante mencionar que cuando está en verde, lo hace a una velocidad cercana a la velocidad máxima a la que puede ir el robot. Por el otro lado, cuando el puzzlebot detecta color amarillo, su velocidad baja, y finalmente, cuando detecta el color rojo se para por completo hasta que vuelve a detectar el color verde, de no ser así, mantiene la velocidad en cero. En caso de no detectar ninguno de los 3 colores, la máquina de estados se queda en el último detectado hasta detectar uno nuevo.

Conclusiones

En este reto se logró implementar un sistema funcional de seguimiento de línea para el Puzzlebot utilizando visión artificial y ROS 2. El robot fue capaz de detectar la trayectoria mediante la cámara, procesar la información visual en la Jetson y utilizarla como referencia para generar los movimientos necesarios de velocidades. Esto permitió que el reto se resolviera de manera correcta y que por ende el Puzzlebot siguiera la línea central de la pista de manera autónoma cumpliendo con el propósito principal del reto.

Consecuentemente, los objetivos planteados se cumplieron en su totalidad ya que se logró integrar una arquitectura compuesta en un nodo controlador y un nodo de visión. El nodo de visión permitió obtener información relevante de la pista como el punto de seguimiento y el lookahead point, la visibilidad de la línea de la pista y el color detectado del semáforo. Por el contrario, el nodo controlador utilizó esta información para calcular las velocidades lineales y angulares enviadas por */cmd_vel*. De igual manera la implementación de un control tipo Pure Pursuit permitió corregir la dirección del robot con base a la diferencia entre el punto objetivo y el centro de la imagen. También se logró integrar la

detección del semáforo dentro del nodo de control a través de una máquina de estados que permitió que el robot respondiera a condiciones visuales del entorno lo que representa un avance importante respecto a una navegación únicamente basada en trayectoria.

De igual manera, el preprocesamiento de la imagen fue algo esencial en el reto para obtener una detección estable y fiable. La selección de región de interés (ROI), la conversión a escala de grises de la ROI, el filtro gaussiano y las operaciones morfológicas de apertura y cierre permitieron reducir el ruido, eliminar detecciones no deseadas y mejorar la continuidad de la línea segmentada. Lo anterior ayudó a que el controlador recibiera una referencia visual más confiable durante el movimiento del robot. Sin embargo, una de las principales limitaciones que se tienen aún en este sistema es que el robot debe iniciar alineado con la línea central de la pista puesto que el controlador asume que existe una referencia visual inicial válida para calcular el punto de seguimiento. Si el robot se coloca demasiado lejos de la línea o con una orientación inicial muy desviada, el sistema puede tardar más en corregirse o incluso perder la referencia. De igual manera, la detección puede verse afectada por cambios fuertes de iluminación en el fondo. Por ende, como posible mejora de la metodología es que se podría calibrar con mayor precisión los umbrales de segmentación para diferentes condiciones de luz.

En conclusión, este reto permitió que se aplicaran de manera integral los conocimientos de visión artificial, preprocesamiento de imágenes, programación con ROS y control de movimiento. El sistema desarrollado para este mini reto cumplió con los requerimientos principales ya que el Puzzlebot logró seguir la pista y responder a señales del semáforo de una gran manera.

Referencias

“Pure Pursuit — Algorithms for Automated Driving.” *Thomasfermi.github.io*, thomasfermi.github.io/Algorithms-for-Automated-Driving/Control/PurePursuit.html.